Digital Twins for the Software Engineer

This chapter introduces the digital twin (DT) to a reader whose training is in software engineering. It assumes familiarity with software architecture, middleware, distributed systems and continuous delivery, and it assumes no prior reading in the digital twin literature. It therefore defines the vocabulary of that literature and leaves the vocabulary of software engineering undefined. What the chapter does not attempt is a survey of application domains or a comparison of commercial platforms; both are deliberately out of scope.

The chapter argues toward one question, which the remainder of the thesis inherits: *what does treating a digital twin as a software-intensive system, rather than as a modelling artefact, demand of software engineering practice?* The question is worth asking because the two framings lead to different work. Read as a modelling artefact, a twin is finished when it predicts well. Read as a software system, it is finished when it can be deployed, observed, changed and retired — and prediction is one property among several.

What the term denotes

The concept originates in product lifecycle management, where [1] formulated it in three parts: a physical product in real space, a virtual product in virtual space, and the data connections tying the two together. That formulation is a useful anchor and a poor specification, because it says nothing about what the virtual product is *for*. [2] extend it to five dimensions by adding data and services as first-class elements, on the argument that the original three cannot accommodate what applications actually require. The addition of a service layer is the step that matters here: it is the point at which the twin acquires consumers, interfaces and availability requirements, and therefore the point at which it becomes software.

The most operationally useful classification for a software engineer is that of Kritzinger et al., which discriminates not by domain but by degree of data integration between the physical and digital counterparts [3]. Where both directions of data flow are manual, the artefact is a *digital model*. Where the flow from physical to digital is automatic but the return path is not, it is a *digital shadow*. Only where both directions are automatic does the term *digital twin* apply. This is a checkable property of a deployed system rather than a claim about its ambition, which is precisely why it is worth adopting.

It would overstate the field's maturity to present that classification as settled. Definitional plurality is documented rather than merely suspected: [4] survey the competing definitions without converging on one, [5] are still proposing a unifying reference model, and [6] report that even within a standardised vocabulary a systematic mapping reached no consensus on type definitions. The practical consequence for a reader of this literature is to treat the label sceptically and read for the integration level instead. In the manufacturing review that proposed the classification, work at the fully integrated level was itself found to be scarce relative to work on digital models and digital shadows [3].

Why this is a software engineering problem

The artefact an engineer must actually build, deploy and keep running is software: models, ingestion pipelines, middleware, storage and the services built on top. That this constitutes

a software engineering concern is not this chapter's proposal but an established framing, mapped across domains by [7] and set out as a community agenda by [8].

Read that way, much of the construction problem is recognisable. Architecture-level technique applies directly, and has been applied: both [9] and [10] address twin construction through architectural patterns and views. The integration layer has been surveyed as middleware in its own right [11], an open-source tooling ecosystem exists and has been compared through case studies [12], and reuse across twins has been pursued as composition on a shared platform [13]. What the twin exists to deliver is likewise a service catalogue — monitoring, prediction, decision support [14] — and twins are rarely deployed alone, which makes them a systems-of-systems integration problem [15], [16].

None of this is exotic to the intended reader. That is the point of the section: the claim is not that digital twins require a new software engineering, but that they exercise the existing one, and that the interesting question is where they exercise it past its usual limits.

Where the familiar toolkit runs out

Three pressure points recur, and they are where this thesis locates its contribution.

First, correctness is defined against a physical referent that the engineer does not control. A conventional system is correct with respect to a specification; a twin is correct with respect to an asset that ages, is repaired and is modified. Verification, validation and uncertainty quantification for twins are consequently identified as open foundational gaps rather than as solved engineering [17]. Model-driven engineering supplies part of the response, with its own limitations acknowledged [18].

Second, time enters the specification. A software engineer normally treats latency as a quality attribute to be budgeted. In a twin whose value depends on tracking a live asset, the divergence between the physical system's timeline and the twin's is a correctness condition, and one that has to be addressed explicitly in the design rather than absorbed [19].

Third, the twin evolves continuously alongside the asset, so there is no release after which it is stable. Evolution has accordingly been argued to sit at the centre of digital twin engineering [20] and has been given explicit notational support [21]. The consequence for practice is a convergence of modelling with operations, pursued as model-based DevOps [22] and, for twins specifically, as TwinOps [23]. A related and less comfortable consequence is that much twin construction remains manual and hard to reproduce [24], which the community has begun to answer with shared exemplars [25].

An open disagreement

The literature does not agree on whether any of this is genuinely new. [26] pose the question directly: is the digital twin an evolution of modelling and simulation, or a revolution? The evolutionary reading is defensible — co-simulation, model calibration and validation all predate the term, and a twin assembled from them inherits their methods. This thesis sides with that evolutionary reading as far as the modelling is concerned, and locates the discontinuity elsewhere: what changes is the obligation to run the model as a live, connected, evolving service against an asset in the field. That obligation is

what sections 1.2 and 1.3 have been describing, and it is enough to make the engineering problem distinct without requiring the stronger claim that the science is.

[1] M. Grieves and J. Vickers, "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems," in *Transdisciplinary Perspectives on Complex Systems: New Findings and Approaches*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds., Cham: Springer International Publishing, 2017, pp. 85–113. doi: 10.1007/978-3-319-38756-7_4.

[2] F. Tao *et al.*, "Five-dimension digital twin model and its ten applications," *Comput. Integr. Manuf. Syst*, vol. 25, no. 1, pp. 1–18, 2019.

[3] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital Twin in manufacturing: A categorical literature review and classification," *Ifac-PapersOnline*, vol. 51, no. 11, pp. 1016–1022, 2018, Accessed: Dec. 11, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896318316021

[4] C. Semeraro, M. Lezoche, H. Panetto, and M. Dassisti, "Digital twin paradigm: A systematic literature review," *Computers in Industry*, vol. 130, p. 103469, Sep. 2021, doi: 10.1016/j.compind.2021.103469.

[5] J. Pfeiffer *et al.*, "Towards a Unifying Reference Model for Digital Twins of Cyber-Physical Systems." arXiv, Jul. 2025. doi: 10.48550/arXiv.2507.04871.

[6] C. Ellwein *et al.*, "Rethinking Asset Administration Shell Communication Types: A Systematic Mapping Study and Portfolio-Based Classification," *Production Engineering*, vol. 20, Dec. 2025, doi: 10.1007/s11740-025-01378-3.

[7] M. Dalibor *et al.*, "A Cross-Domain Systematic Mapping Study on Software Engineering for Digital Twins," *Journal of Systems and Software*, vol. 193, p. 111361, Nov. 2022, doi: 10.1016/j.jss.2022.111361.

[8] L. Cleophas *et al.*, "A community-sourced view on engineering digital twins: A report from the EDT.Community," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, in MODELS '22. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 481–485. doi: 10.1145/3550356.3561549.

[9] B. Tekinerdogan and C. Verdouw, "Systems Architecture Design Pattern Catalog for Developing Digital Twins," *Sensors*, vol. 20, no. 18, p. 5103, Jan. 2020, doi: 10.3390/s20185103.

[10] E. Ferko, A. Bucaioni, and M. Behnam, "Architecting Digital Twins," *IEEE Access*, vol. 10, pp. 50335–50350, 2022, doi: 10.1109/ACCESS.2022.3172964.

[11] A. Almeida, T. Batista, E. Cavalcante, F. Delicato, R. Motta, and M. Vieira, "Middleware for Digital Twins: A Systematic Mapping Study," in *Proceedings of the 1st International Workshop on Middleware for Digital Twin*, in Midd4DT '23. New York, NY, USA: Association for Computing Machinery, Dec. 2023, pp. 19–24. doi: 10.1145/3631319.3632302.

[12] S. Gil, P. H. Mikkelsen, C. Gomes, and P. G. Larsen, "Survey on open-source digital twin frameworks–A case study approach," *Software: Practice and Experience*, vol. 54, no. 6, pp. 929–960, Jun. 2024, doi: 10.1002/spe.3305.

[13] P. Talasila *et al.*, "Composable digital twins on Digital Twin as a Service platform," *SIMULATION*, vol. 101, no. 3, pp. 287–311, Mar. 2025, doi: 10.1177/00375497241298653.

[14] M. Frasheri, T. Böttjer, P. G. Larsen, L. Esterle, and C. Gomes, "Advanced Digital Twin Services," in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 209–222. doi: 10.1007/978-3-031-66719-0_10.

[15] J. Michael, J. Pfeiffer, B. Rumpe, and A. Wortmann, "Integration challenges for digital twin systems-of-systems," in *Proceedings of the 10th IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems*, in SESoS '22. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 9–12. doi: 10.1145/3528229.3529384.

[16] B. Combemale *et al.*, "On the Challenges of Integrating Digital Twins," in *2nd International Conference on Engineering Digital Twins (EDTconf 2025)*, 2025. Accessed: Sep. 29, 2025. [Online]. Available: https://inria.hal.science/hal-05221809/

[17] *Foundational Research Gaps and Future Directions for Digital Twins*. Washington, D.C.: National Academies Press, 2024. doi: 10.17226/26894.

[18] J. Michael *et al.*, "Model-Driven Engineering for Digital Twins: Opportunities and Challenges," *Systems Engineering*, vol. 28, no. 5, pp. 659–670, 2025, doi: 10.1002/sys.21815.

[19] M. Frasheri *et al.*, "Addressing time discrepancy between digital and physical twins," *Robotics and Autonomous Systems*, vol. 161, p. 104347, Mar. 2023, doi: 10.1016/j.robot.2022.104347.

[20] T. Alskaif *et al.*, "Evolution at the Core of Digital Twin Engineering," Grand Rapids, USA: IEEE, Jul. 2025. doi: 10.5283/epub.77656.

[21] J. Mertens, S. Klikovits, F. Bordeleau, J. Denil, and Ø. Haugen, "Continuous Evolution of Digital Twins using the DarTwin Notation." arXiv, Oct. 2024. Accessed: Nov. 11, 2024. [Online]. Available: http://arxiv.org/abs/2410.23389

[22] B. Combemale *et al.*, "Model-Based DevOps: Foundations and Challenges," in *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, Oct. 2023, pp. 429–433. doi: 10.1109/MODELS-C59198.2023.00076.

[23] J. Hugues, J. Yankel, J. Hudak, and A. Hristozov, "Twinops: Digital twins meets devops," *CARNEGIE-MELLON UNIV PITTSBURGH PA, Tech. Rep.*, 2022, Accessed: Jan. 08, 2025. [Online]. Available: https://apps.dtic.mil/sti/trecms/pdf/AD1168471.pdf

[24] A. Barbie and W. Hasselbring, "Toward Reproducibility of Digital Twin Research: Exemplified with the PiCar-X." arXiv, Aug. 2024. doi: 10.48550/arXiv.2408.13866.

[25] E. Kamburjan *et al.*, "GreenhouseDT: An Exemplar for Digital Twins," in *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, in SEAMS '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 175–181. doi: 10.1145/3643915.3644108.

[26] Z. Ali, R. Biglari, J. Denil, J. Mertens, M. Poursoltan, and M. K. Traoré, "From modeling and simulation to Digital Twin: Evolution or revolution?" *SIMULATION*, vol. 100, no. 7, pp. 751–769, Jul. 2024, doi: 10.1177/00375497241234680.